matplotlib

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:
```
#解决中文显示问题
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False
```

对象容器

1. Figure: 负责图形大小
2. Axes: 负责坐标轴位置，绘图等
3. Axis: 负责坐标轴设置
4. Tick: 负责格式化刻度的样式

plot函数调用格式

plot(x, y, linestyle, linewidth, color, marker, markersize,
markeredgecolor, markerfacecolor, markeredgewidth, label, alpha)

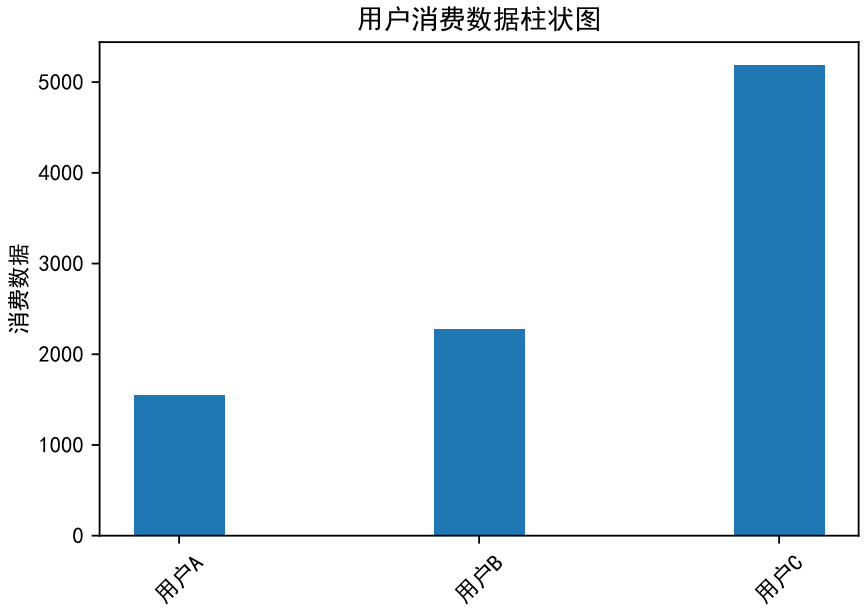
linestyle: 指定折线的类型，实线、虚线、点画线
linewidth: 折线的宽度
marker: 可以为折线图添加点，该参数设置点的形状
markersize: 设置点的大小
markeredgecolor: 设置点的边框颜色
markerfacecolor: 设置点的填充颜色
markeredgewidth: 设置点的边框宽度
label: 添加折线图的标签，类似于图例的作用
alpha: 设置图形的透明度

matplotlib.pyplot模块其他绘图函数

pie(): 饼图
bar(): 柱状图 hist(): 二维直方图 scatter(): 散点图

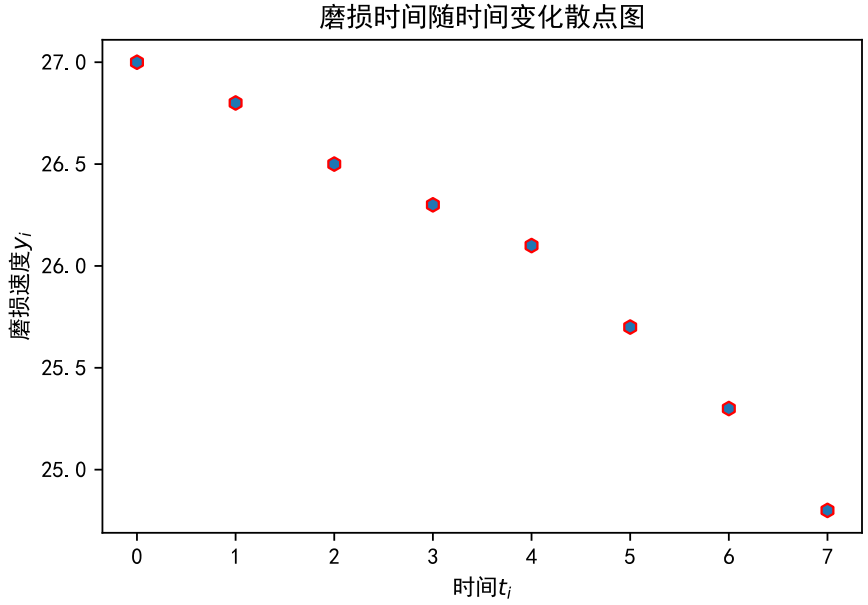
调用plt.show()显示已绘图像

In [3]:
```
#绘制柱状图，使用cgxf_pd2.xlsx中数据
a_data = pd.read_excel(io='cgxf_pd2.xlsx', engine='openpyxl', usecols=range(1, 4))
a_sum = np.sum(a_data, axis=0)
plt.bar(x=[1, 2, 3], height=a_sum, width=0.3)
plt.ylabel("消费数据")
plt.xticks([1, 2, 3], ["用户A", "用户B", "用户C"], rotation=45)
plt.title("用户消费数据柱状图")
plt.show()
```



In [4]:

```
#散点图
x = np.arange(8)
y = '27 26.8 26.5 26.3 26.1 25.7 25.3 24.8'
#将数据间空格转换成逗号
y = ','.join(y.split(sep=" ", maxsplit=y.count(" ")))
y = np.array(eval(y))
plt.scatter(x, y, marker='h', edgecolors='r')
plt.xlabel("时间$t_i$")
plt.ylabel("磨损速度$y_i$")
plt.title("磨损时间随时间变化散点图")
plt.show()
```



'sep'.join(seq) 返回一个以分隔符sep连接各个元素后生成的字符串

sep: 分隔符。可以为空

seq: 要连接的元素序列、字符串、元组、字典

split()通过指定分隔符对字符串进行切片，如果第二个参数maxsplit有指定值，则分割为maxsplit+1个子字符串

str.split(sep="", maxsplit=string.count(str))

str: 分隔符，默认为所有的空字符，包括空格、换行(\n)、制表符(\t)等

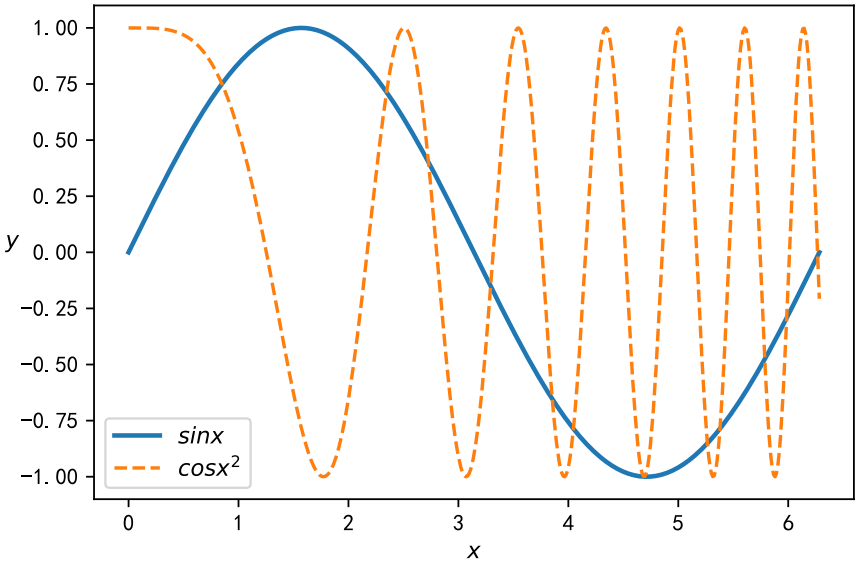
num: 分割次数。默认为 -1, 即分隔所有

eval()函数用来执行一个字符串表达式，并返回表达式的值

多个图形显示在一张图上

在一张图上画出$y = sinx$，　$y = cosx^2$，　$x \in [0, 2\pi]$

In [5]:
```
x = np.linspace(0, 2*np.pi, 500)
y1 = np.sin(x)
y2 = np.cos(x**2)
plt.plot(x, y1, label='$sinx$', linewidth=2)
plt.plot(x, y2, label='$cosx^2$', linestyle='--')
plt.xlabel("$x$")
plt.ylabel("$y$", rotation=0)
plt.legend()
plt.show()
```

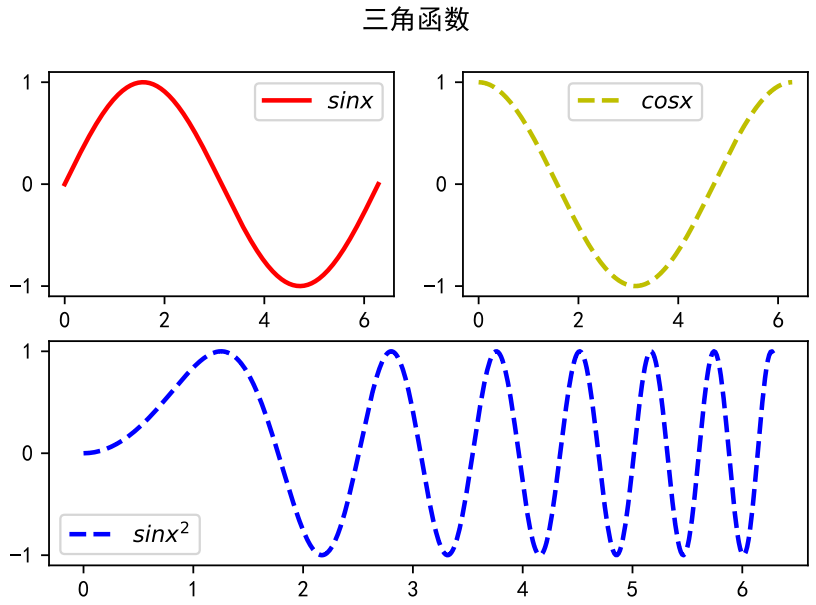


多个图单独显示

将屏幕开成三个子窗口，上两个，下一个，分别画

$y = sinx$　　$y = cosx$　　$y = sinx^2$　　$x \in [0, 2\pi]$

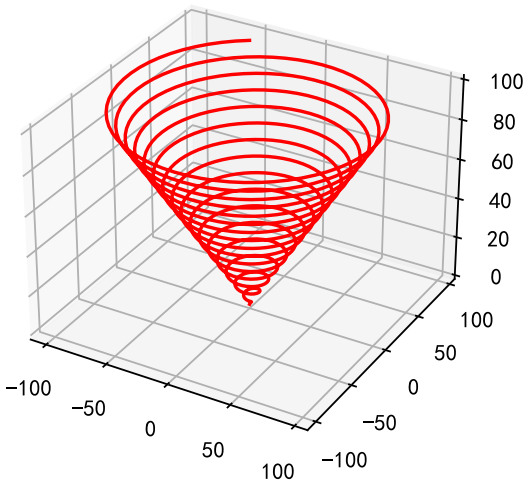
In [6]:
```
x = np.linspace(0, 2*np.pi, 500)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(pow(x, 2))

ax1 = plt.subplot(2, 2, 1)
ax1.plot(x, y1, linestyle='-', color='r', linewidth=2.0, label='$sinx$')
ax1.legend()
ax2 = plt.subplot(2, 2, 2)
ax2.plot(x, y2, linestyle='--', color='y', linewidth=2.0, label='$cosx$')
ax2.legend()
ax3 = plt.subplot(2, 1, 2)
ax3.plot(x, y3, linestyle='--', color='b', linewidth=2.0, label='$sinx^2$')
ax3.legend()
plt.suptitle("三角函数")
plt.show()
```



三维空间曲线

画出三维曲线$x = tsint, \quad y = tcost, \quad z = t \qquad t \in [0, 100]$图像

In [7]:
```
#设置三维图形模式
ax = plt.axes(projection='3d')
z = np.linspace(0, 100, 1000)
x = z * np.sin(z)
y = z * np.cos(z)
ax.plot3D(x, y, z, 'r-')
plt.show()
```



三维空间曲面

绘制三维曲面$z = sin(\sqrt{x^2 + y^2})$的表面图和三维网格图

In [8]:
```
# np.meshgrid()函数 (生成网格点坐标矩阵)
# 即把两个数组的笛卡尔积内的元素的第一、二个坐标分别放入两个矩阵中
x = np.array([0, 1, 2])
y = np.array([3, 4, 5])

X, Y = np.meshgrid(x, y)
print(X, '\n')
print(Y)
```

```
[[0 1 2]
 [0 1 2]
 [0 1 2]]

[[3 3 3]
```

```
 [4 4 4]
 [5 5 5]]
```

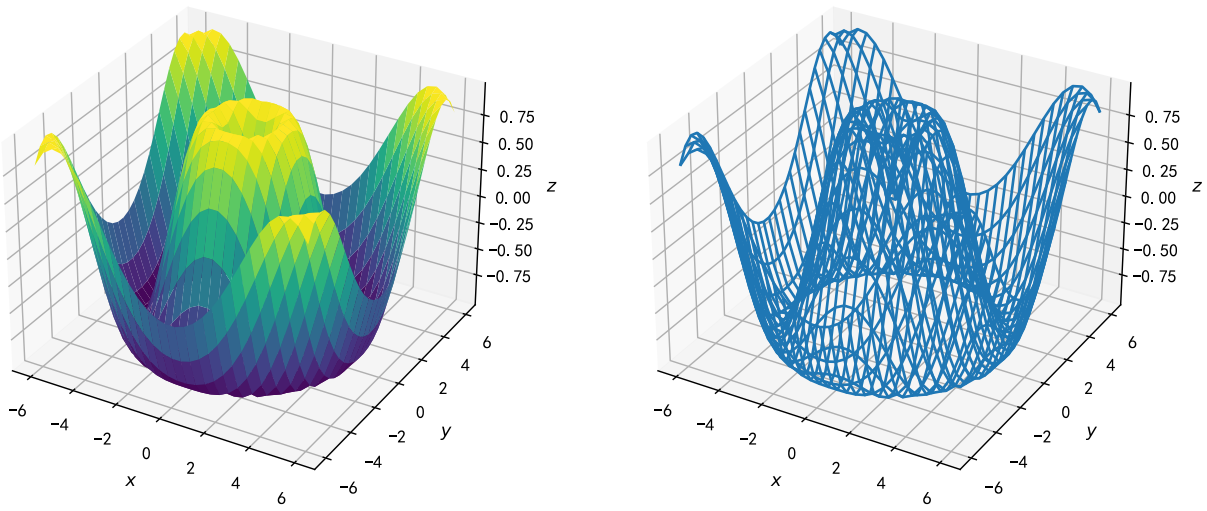
笛卡尔积

$\mathbf{a} = [0, 1, 2] \qquad \mathbf{b} = [3, 4, 5]$

$\mathbf{a} \times \mathbf{b} = \{< 0, 3 >, < 0, 4 >, < 0, 5 >, < 1, 3 >, < 1, 4 >, < 1, 5 >, < 2, 3 >, < 2, 4 >, < 2, 5 >\}$

In [9]:

```
#绘图
x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)     #生成网格点坐标矩阵
Z = np.sin(np.sqrt(X**2 + Y**2))

plt.figure(figsize=(12,16))     #设置绘图区的大小
# 表面图
ax1 = plt.subplot(121, projection='3d')
ax1.plot_surface(X, Y, Z, cmap='viridis')
ax1.set_xlabel('$x$')
ax1.set_ylabel('$y$')
ax1.set_zlabel('$z$')
# 网格图
ax2 = plt.subplot(122, projection='3d')
ax2.plot_wireframe(X, Y, Z)
ax2.set_xlabel('$x$')
ax2.set_ylabel('$y$')
ax2.set_zlabel('$z$')
plt.show()
```



plot_surface() 绘制3D曲面

rstride: 行之间的跨度
cstride: 列之间的跨度
rcount: 设置间隔个数，默认50个
ccount: 列的间隔个数 不能与上面两个参数同时出现
cmap: 是颜色映射表
zdir: 可选 'z'|'x'|'y'| 分别表示投影到z,x,y平面
zdir = 'z', offset = -2 表示投影到z = -2上

ax.plot_surface(X, Y, Z, rstride = 1, cstride = 1, cmap = cm.coolwarm)

plot_wireframe() 绘制3D线框图(参数与上基本相同)

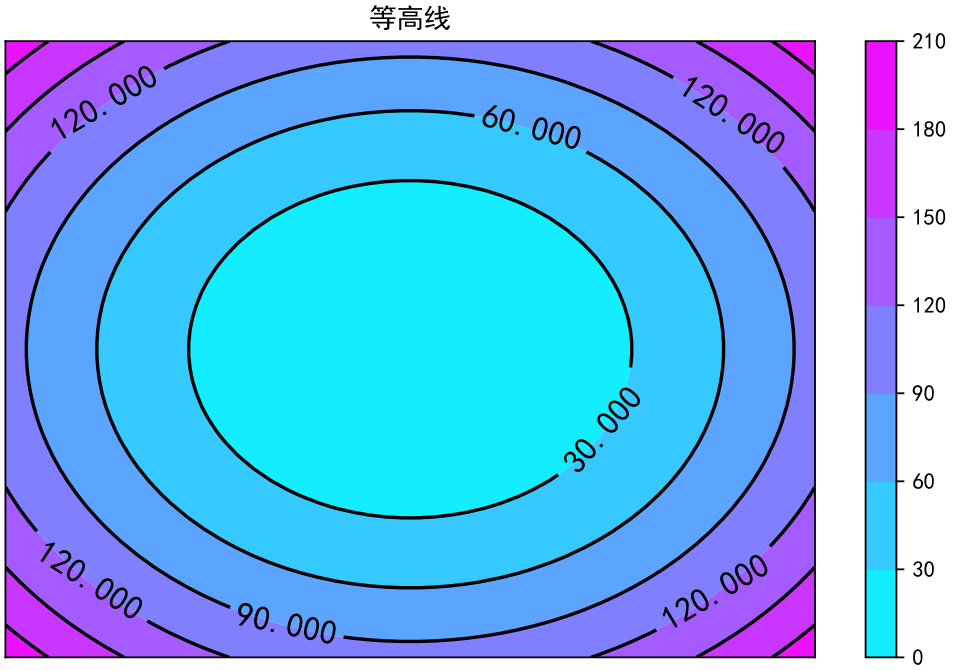
等高线图

以$z = x^2 + y^2$为例

In [10]:

```
#取点
x = np.linspace(-10, 10, 50)
y = np.linspace(-10, 10, 50)
```

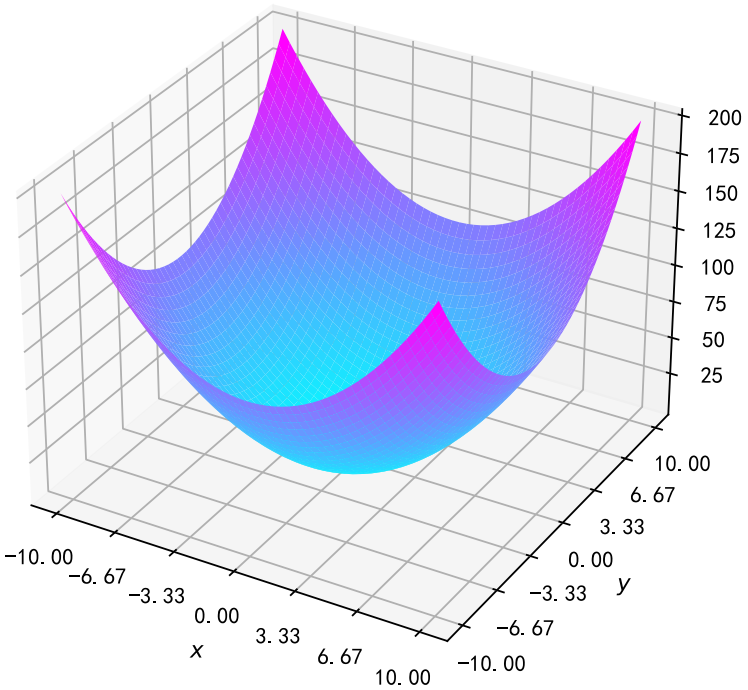
```
#组成网格图
X, Y = np.meshgrid(x, y)
Z = X**2 + Y**2
#设置画板大小
plt.figure(figsize=(8, 5))
#填充颜色，f即filled, levels是颜色层数
cor = plt.contourf(X, Y, Z, levels=6, cmap='cool')
#画等高线
contr = plt.contour(X, Y, Z, levels=6, colors='k')
#在等高线上标明高度值
plt.clabel(CS=contr, fontsize=15, colors='k')
#删去坐标轴
plt.xticks(())
plt.yticks(())
plt.title('等高线')
#添加颜色条
plt.colorbar(mappable=cor)
#显示
plt.show()
```



In [11]:

```
#绘制三维图与等高图对比
plt.figure(figsize=(9, 6))
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, cmap='cool')
#设置坐标轴
plt.xticks(np.linspace(-10, 10, 7))
plt.yticks(np.linspace(-10, 10, 7))

plt.xlabel('$x$')
plt.ylabel('$y$')
plt.title('$z = x^2 + y^2$')
plt.show()
```

$$z = x^2 + y^2$$

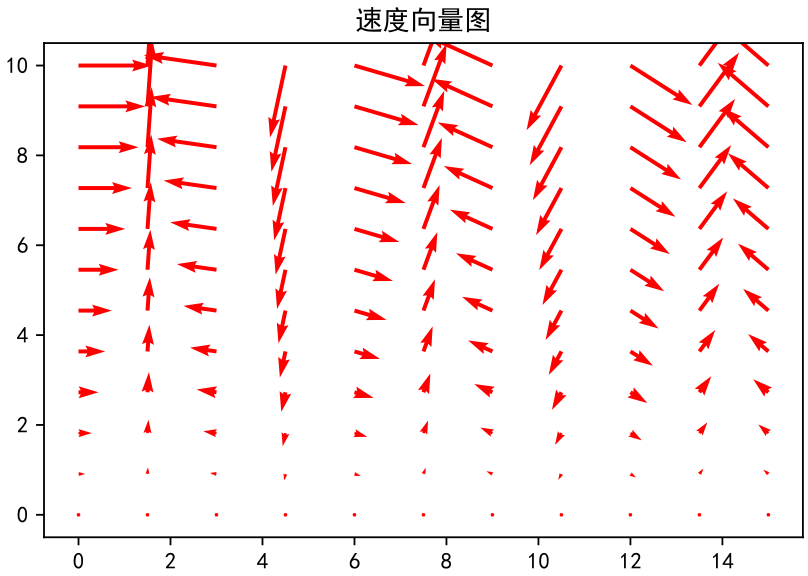


向量图

绘出速度向量$(u, v) = (ycosx, ysinx)$的向量场

In [12]:

```
x = np.linspace(0, 15, 11)
y = np.linspace(0, 10, 12)
#生成网格点
X, Y = np.meshgrid(x, y)
#速度向量
u = Y * np.cos(X)
v = Y * np.sin(X)
#向量图
plt.quiver(X, Y, u, v, color='r')
plt.title('速度向量图')
plt.show()
```



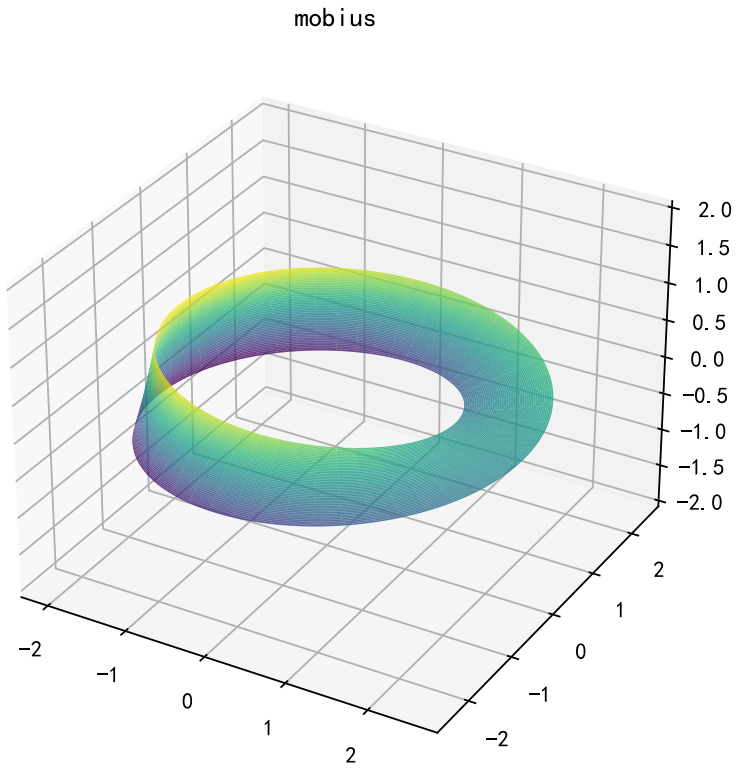
莫比乌斯带

参数方程为:

$$\begin{cases} x = (2 + \frac{s}{2}cos\frac{t}{2})cost \\ y = (2 + \frac{s}{2}cos\frac{t}{2})sint \\ z = \frac{s}{2}sin\frac{t}{2} \end{cases} \qquad t \in [0, 2\pi], \quad s \in [-1, 1]$$

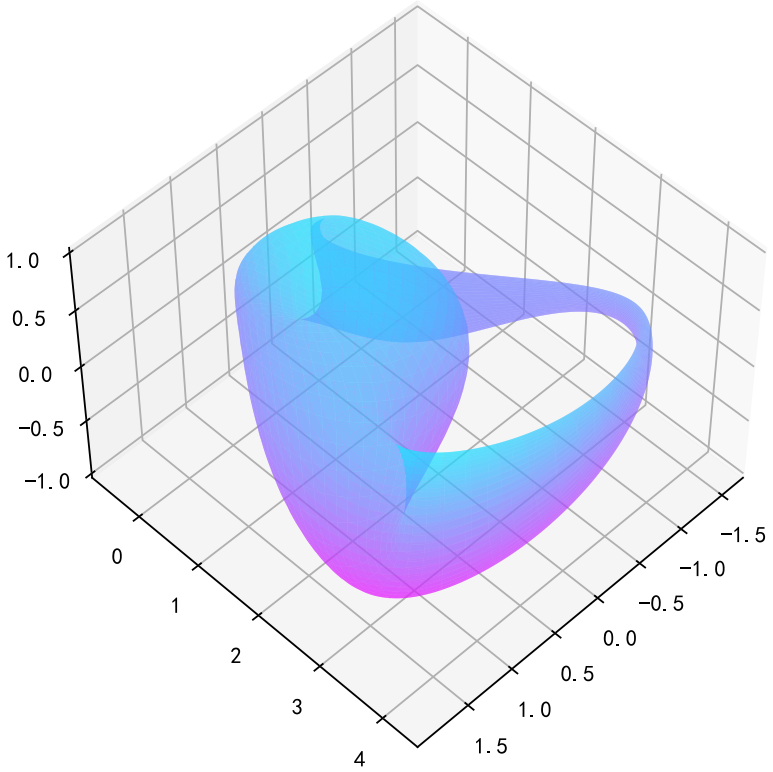
In [13]:

```
#取点
t = np.linspace(0, 2*np.pi, 500)
s = np.linspace(-1, 1, 500)
#网格点
T, S = np.meshgrid(t, s)

x = (2 + S/2 * np.cos(T/2)) * np.cos(T)
y = (2 + S/2 * np.cos(T/2)) * np.sin(T)
z = S/2 * np.sin(T/2)

plt.figure(figsize=(10, 6))
ax = plt.axes(projection='3d')
ax.plot_surface(x, y, z, cmap='viridis', alpha=0.8)
#拉长z坐标轴
ax.set_zticks(np.linspace(-2, 2, 9))
ax.set_title('mobius')
plt.show()
```



In [14]:

```
#klein瓶
u = np.linspace(0, np.pi, 500)
v = np.linspace(0, 2 * np.pi, 500)
U, V = np.meshgrid(u, v)

x = -2/15*np.cos(U)*(3*np.cos(V)-30*np.sin(U)+90*(np.cos(U))**4*np.sin(U)-60*(np.cos(U))**6*np.sin(U
y = -1/15*np.sin(U)*(3*np.cos(V)-3*(np.cos(U))**2*np.cos(V)-48*(np.cos(U))**4*np.cos(V)-60*np.sin(U)
z = 2/15*(3+5*np.cos(U)*np.sin(U))*np.sin(V)

plt.figure(figsize=(10, 6))
ax = plt.axes(projection='3d')
ax.plot_surface(x, y, z, cmap='cool_r', alpha=0.5)
ax.set_zticks(np.linspace(-1, 1, 5))
#调整视角
ax.view_init(elev=50, azim=45)
plt.show()
```



In [ ]: